流类库与输入/输出

流类库与输入输出控制

- C++流类体系
- 输入输出的格式控制
- 标准设备的输入输出
- 文件的输入输出
- 文件与对象

流类库与输入/输出

- 输入/输出是什么？

  Input/output (I/O)程序与外设之间的信息交换。

- 输入数据的来源地，如键盘、鼠标、磁盘文件等

- 接受输出数据的地方，如磁盘文件、显示器、打印机等

- 流**stream**：字节序列在源、程序、目的之间的流动。（文本流和二进制流）

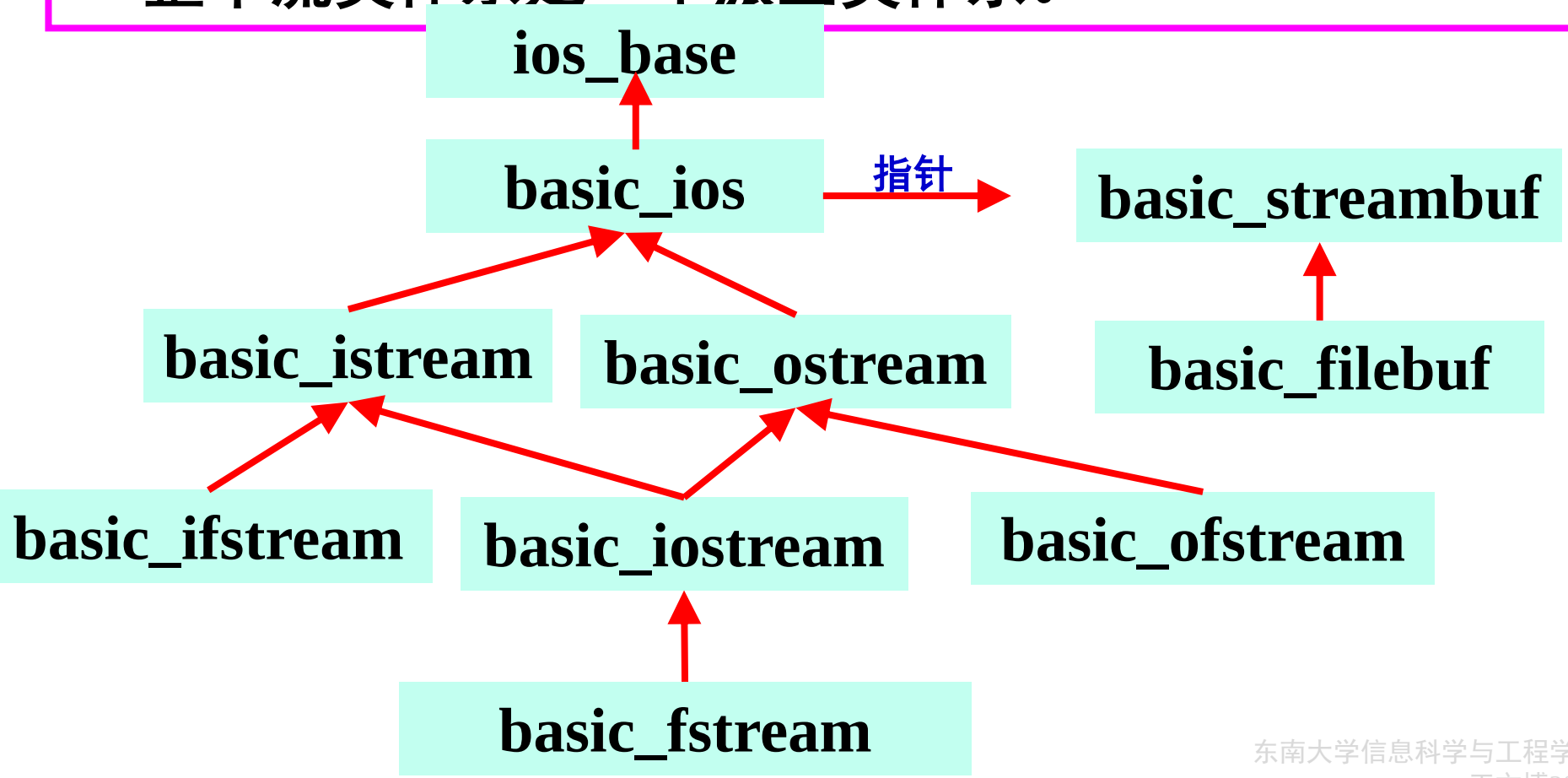
- 流类：

  实现**I/O**操作的一组类组成类库。

东南大学信息科学与工程学院
王文博2026

C++的基本流类体系

- C++语言中并没有输入/输出语句，而是在标准库里包含了一个I/O流类库。
- 整个流类体系是一个派生类体系。



C++的基本流类体系

- 标准I/O(键盘<->显示器): <iostream>

  (包含<ios>、<streambuf>、<istream> 和 <ostream>)

- 输入/输出文件流：<fstream> 。

- 读操作在流数据抽象中被称为（从流中）提取>>，
- 写操作被称为（向流中）插入<<。

东南大学信息科学与工程学院
王文博2026

C++的基本流类体系

iostream中预定义的标准流对象

istream对象

otream对象

cin	有缓冲	键盘输入
cout	有缓冲	一般信息，到显示器
cerr	无缓冲	错误信息，到显示器
clog	有缓冲	日志信息，到显示器

为提高输入/输出效率，常用缓冲区：内存中的一块区域，输入/输出先放在缓冲区中，当缓冲满或输入/输出遇到换行符时才将缓冲的内容做一次性处理（输出或传递给用户）。这样的流称为缓冲流。若对输入/输出立即响应则为非缓冲的I/O操作。

东南大学信息科学与工程学院
王文博2026

2 输入输出格式的控制

为了控制输出的美观和阅读方便，C++的流类库给出了格式化的I/O流。

xiosbase中定义的格式控制符

```
enum _Fmtflags
{   // constants for formatting options
        _Fmtmask = 0xffff, _Fmtzero = 0};
static const _Fmtflags skipws = (_Fmtflags)_IOSskipws;
static const _Fmtflags unitbuf = (_Fmtflags)_IOSunitbuf;
static const _Fmtflags uppercase = (_Fmtflags)_IOSuppercase;
static const _Fmtflags showbase = (_Fmtflags)_IOSshowbase;
static const _Fmtflags showpoint = (_Fmtflags)_IOSshowpoint;
static const _Fmtflags showpos = (_Fmtflags)_IOSshowpos;
static const _Fmtflags left = (_Fmtflags)_IOSleft;
static const _Fmtflags right = (_Fmtflags)_IOSright;
static const _Fmtflags internal = (_Fmtflags)_IOSinternal;
static const _Fmtflags dec = (_Fmtflags)_IOSdec;
static const _Fmtflags oct = (_Fmtflags)_IOSoct;
```

了解

东南大学信息科学与工程学院 王文博 2026

2 输入输出格式的控制

```
static const _Fmtflags hex = (_Fmtflags)_IOShex;
static const _Fmtflags scientific = (_Fmtflags)_IOSscientific;
static const _Fmtflags fixed = (_Fmtflags)_IOSfixed;
static const _Fmtflags boolalpha = (_Fmtflags)_IOSboolalpha;
static const _Fmtflags _Stdio = (_Fmtflags)_IOS_Stdio;
static const _Fmtflags adjustfield = (_Fmtflags)(_IOSleft
                                    | _IOSright | _IOSinternal);
static const _Fmtflags basefield = (_Fmtflags)(_IOSdec
                                     | _IOSoct | _IOShex);
static const _Fmtflags floatfield = (_Fmtflags)(_IOSscientific
                                     | _IOSfixed);
```

_Fmtmask

_Stdio	boolalpha	fixed	scientific	hex	oct	dec	internal

right	left	showpos	showpoint	showbase	uppercase	unitbuf	skipws

了解

东南大学信息科学与工程学院 王文博 2026

2 输入输出格式的控制

标志字均为保护成员，需通过接口函数访问：

```
inline long ios::flags() const ;
inline long ios::flags(long _l);
inline long ios::setf(long _l,long _m);
inline long ios::setf(long _l);
inline long ios::unsetf(long _l);
```

返回当前标志字

把参数当做新的标志字，返回原标志字

在原标志字上增加控制

增加控制项（多项）

清除指定位上的控制

了解

东南大学信息科学与工程学院
王文博2026

2 输入输出格式的控制

```
inline int ios::width() const ;
```
返回当前域宽

```
inline int ios::width(int);
```
把参数当做新的域宽，返回原域宽

```
inline char ios::fill() const;
```
返回当前填充字符

```
inline char ios::fill(char);
```
参数做新填充字符，返回原字符

```
inline int ios::precision(int);
```
参数作为新精度，返回原精度

```
inline  int ios:: precision() const;
```
返回当前精度

东南大学信息科学与工程学院
王文博2026

2 输入输出格式的控制

```
#include <iostream>
using namespace std;
int main（ ）
{int a=21
 cout.setf（ios::showbase）;
 cout<<"dec:"<<a<<endl;
cout.unsetf（ios::dec）
 cout.setf（ios::hex）;
cout<<"hex:"<<a<<endl;
cout.unsetf（ios::hex）;
cout.setf（ios::oct）;
cout<<"oct:"<<a<<endl;
cout.unseft（ios::oct）;
char *pt="China";
cout.width（10）;
cout<<pt<<endl;
cout.width（10）;
 cout.fill（'*'）;
cout<<pt<<endl;
double pi=22.0/7.0;
cout.setf（ios::scientific）;
cout<<"pi=";
cout.width（14）;
cout<<pi<<endl;
cout.unsetf（ios::scientific）;
cout.setf（ios::fixed）;
cout.width（12）;
cout.setf（ios::showpos）;
cout.setf（ios::internal）;
cout.precision（6）;
cout<<pi<<endl;
return 0;
}
```

显示基数符号（0x或0）

dec:21

hex:0x15

oct:025

China

*****China

科学记数法

**3.142857e+00

+***3.142857

2 输入输出格式的控制

流格式控制成员函数使用比较麻烦，用流操作子（控制符）比较简洁。

ios中

fx **Format flag manipulators (functions)**

Independent flags (switch on):	
boolalpha	Alphanumerical bool values (function)
showbase	Show numerical base prefixes (function)
showpoint	Show decimal point (function)
showpos	Show positive signs (function)
skipws	Skip whitespaces (function)
unitbuf	Flush buffer after insertions (function)
uppercase	Generate upper-case letters (function)
Independent flags (switch off):	
noboolalpha	No alphanumerical bool values (function)
noshowbase	Do not show numerical base prefixes (function)
noshowpoint	Do not show decimal point (function)
noshowpos	Do not show positive signs (function)
noskipws	Do not skip whitespaces (function)
nounitbuf	Do not force flushes after insertions (function)
nouppercase	Do not generate upper case letters (function)
Numerical base format flags ("basefield" flags):	
dec	Use decimal base (function)
hex	Use hexadecimal base (function)
oct	Use octal base (function)
Floating-point format flags ("floatfield" flags):	
fixed	Use fixed floating-point notation (function)
scientific	Use scientific floating-point notation (function)
Adustment format flags ("adjustfield" flags):	
internal	Adjust field by inserting characters at an internal position (function)
left	Adjust output to the left (function)
right	Adjust output to the right (function)

```
#include<iomanip>
```

setfill（ch）
setprecision(n)
setw(n)
setbase(n)

王文博2026

2 输入输出格式的控制

```
#include <iostream>
#include <iomanip>
using namespace std;
int main（）
{   int a=21;
    cout<<"dec:"<<dec<<a<<endl;
    cout<<"hex:"<<hex<<showbase<<a<<endl;
    cout<<"oct:"<<oct<<a<<endl;
    char *pt="China";
    cout<<setw（10）<<pt<<endl;
    cout<<setfill（'*'）<<setw（10）<<pt<<endl;
    double pi=22.0/7.0;
    cout<<scientific<<setprecision（8）;
    cout<<"pi="<<pi<<endl;
    cout<<setprecision（4）<<pi<<endl;
    cout<<fixed<<setw(10)<<internal<<showpos<<pi<<endl;
 return 0;
}
```

```
dec:21
hex:0x15
oct:025
      China
*****China
pi=3.14285714e+000
3.1429e+000
+***3.1429
```


东南大学信息科学与工程学院
王文博2026

3 提高标准输入/输出

- cin为缓冲流，键盘输入的数据保存在缓冲区中，如果一次输入过多，会留在缓冲区待下一次提取。只有把缓冲区的数据取完，才会要求输入新的数据。
- 输入时如果输入错误，须在回车前修改，按下回车后就无法修改了。
- 输入的数据类型必须与提取的数据类型一致，否则会出错。

东南大学信息科学与工程学院
王文博2026

3 提高标准输入/输出

- 标准设备的输入并不安全，稳健性有待提高。
- 可采取的方式：避免输入失误(如类型，个数不匹配)。
- 要解决的问题：如何在程序中发现输入失误，并采取补救措施？

可通过利用状态字state来提高稳健性

东南大学信息科学与工程学院
王文博2026

3 提高标准输入/输出

```
enum _Iostate
{     // constants for stream states
_Statmask = 0x17};


static const _Iostate goodbit = (_Iostate)0x0;
static const _Iostate eofbit = (_Iostate)0x1;
static const _Iostate failbit = (_Iostate)0x2;
static const _Iostate badbit = (_Iostate)0x4;
static const _Iostate _Hardfail = (_Iostate)0x10;
```

流正常

输入流结束，忽略后继提取操作；或文件结束，已无数据可取

最近I/O操作失败；

最近I/O操作非法；

东南大学信息科学与工程学院
王文博2026

3 提高标准输入/输出

ios类内跟state有关的操作

```
int rdstate() const; //读取状态字
void clear(int _i = 0); //人工设置状态, 用来清状态
bool good() const; //正常返回1,否则返回0
bool  eof() const; //返回流(文件)结束位
bool  fail() const; //返回操作非法和操作失败这两位
bool bad() const; //返回非法操作位
bool     operator!() const; //重载操作符“!”以代替fail()
```


东南大学信息科学与工程学院
王文博2026

3 提高标准输入/输出

出错会在流的状态字state对应的位置1。要提高稳健性，需要在编程中加入对状态字的判断。

```
int main()
{
    char str[256];
    int i;
    cout<<"请输入整数:"<<endl;
    cin>>i;
    cout<<"状态字为： "<<cin.rdstate()<<endl;
```

如果输入的是字符呢

0为正常，非0则失败

```
    while( cin.fail() )
    {
      cin.clear(0);
     cin.getline(str,255);
     cout<<"输入错误,请重新输入整数"<<endl;
     cin>>i;
     cout<<"状态字为： "<<cin.rdstate()<<endl; }
```

读取成功返回0，失败返回非0。

流的状态字state不会自动清除，必须通过软件清0。

该语句起什么作用？

3 提高标准输入/输出

```
// 续前页 1
 char c;
   cout<<"请输入字符:"<<endl;
   cin>>c;
   cout<<"输入整数为:"<<i<<endl;
   cout<<"输入字符为:"<<c<<endl;
  return 0;
}
```

两段代码一样吗？

```
// 续前页 2
 ar c1;
cout<<"请输入字符:"<<endl;
cin.get(c1);
cout<<"输入整数为:"<<i<<endl;
cout<<"输入字符为:"<<c1<<endl;
 return 0;
}
```

```
// 续前页3
    cin.getline(str,256);
      cout<<"请输入字符串"<<endl;
      cin.getline(str,255);
      cout<<"输入整数为:"<<i<<endl;
      cout<<"输入字符为:"<<c<<endl;
      cout<<"输入字符串为:"<<str<<endl;
      return 0;
}
```

读空缓冲区,吃掉回车

东南大学信息科学与工程学院
王文博2026

3 提高标准输入/输出

- 使用>>提取输入数据时，空格和回车都可以作为数据之间的分隔符，对于字符型和字符串，空格和回车无法读入。

- 输入数以后再输入字符或字符串时，如果数后直接加回车，应用cin.get()提取回车。 如果还有空格，需要读空缓冲区。

东南大学信息科学与工程学院
王文博2026

4. 输入输出成员函数

get函数

不带参数： `int get();`

调用举例：
```
i=cin.get();
```

提取一个字符，包括空格、回车及制表等

带一个参数：

```
istream& get(char &);
istream& get(unsigned char &);
```

调用举例：
```
char ch;
cin.get(ch);
```

提取一个字符，包括空格、回车及制表等，放在字符型变量中。
读取成功函数返回非0值，如失败，函数返回0值

带三个参数：

```
istream& get(char *,int,char ='\n');
istream& get(unsigned char *,int,char ='\n');
```

调用举例：
```
char ch[20];
cin.get(ch,10 , 'z');
```

注意，不提取结束字符

提取字符串，直到串结束符或指定长度(n,取n-1个字符)。
读取成功函数返回非0值，如失败，函数返回0值

4. 输入输出成员函数

getline函数

如果遇到指定结束符，提取该结束符，但不保存在串中。

istream& getline(char *,int,char ='\n');

istream& getline(unsigned char *,int,char ='\n');

提取字符串，直到串结束符或指定长度(n,取n-1个字符)。

读取成功函数返回非0值，如失败，函数返回0值。

调用示例

```
char ch[20];
cin.getline(ch,20,'X');
```

read函数

istream& read(char *, int);

调用示例

```
cin.read(ch,20);
```

读取指定数量的字符/指定字节数的二进制数据

工程学院
王文博2026

4. 输入输出成员函数

ignore函数

iostream中EOF=-1

istream& ignore(int=1，int=EOF）;

第1个参数为提取的字符数量，第2个字参数表示遇到该字符结束提取该结束字；

功能：空读-对提取的字符不保存、不处理。 可用来消除多余字符和回车符

调用示例

```
ignore（5, 'A'）；
ignore（）；
```

gcount函数

istream& gcount() const{ return x_gcount};

返回最后一次提取的字符数量，包括回车。

工程学院
文博2026

4. 输入输出成员函数

putback函数

istream& putback(char）;

将最后一次从输入流中得到的字符放回输入流中

调用示例

cin.putback（ch）

peek函数

int peek();

读输入流当前字符的值，但是不提它，该字符仍然在输入流当前字符位置上，下次提取的字符还是它

程学院
王文博2026

4. 输入输出成员函数

put函数

ostream& put （char）;

输出一个字符

调用示例

```
cout.put（ 'a' ）;
cout.put（ 97）;
```

write函数

ostream& write （char*, int）;

输出指定数量的字符/指定字节数量的二进制数据

调用示例

```
char *s=“hello”;
cout.write(s,3);
```

大学信息科学与工程学院
王文博2026

4. 输入输出成员函数

```
int main()
{
    char c;
    cout<<"enter a sentence:"<<endl;
    while((c=cin.get())!=EOF)  cout<<c;
    cout<<"状态字：  "<<cin.rdstate()<<endl;
    cout<<"c="<<(int)c<<endl;
    cin.clear(0);
    cout<<cin.rdstate()<<endl;
    char ch;
    cout<<"enter a sentence:"<<endl;
    while(cin.get(ch))   cout<<ch;
    cout<<"状态字：  "<<cin.rdstate()<<endl;
    cin.clear(0);
   cout<<cin.rdstate()<<endl<<endl;
```

键盘输入Ctrl+Z结束输入

输入流最后提取到的是结束符-1

输入流状态字不会自动清0，需要手动设置，不清除的话，读到结束符后面的流对象操作无效

读取成功返回非零，读取失败返回0

东南大学信息科学与工程学院
王文博2026

4. 输入输出成员函数

```
char carr[20];
cout<<"enter a sentence:"<<endl;
cin.get(carr,10,'X');
cout<<"读入字符串为："<<carr<<endl;
cout<<"读入字符串长度为："<<cin.gcount()<<endl;
cout<<"继续读入字符："<<char(cin.get())<<endl;
```

指定字符X为结束符

如果输入：Happy New Year!

三个输出语句输出什么？

输入：THE BOX IS BIG! 输出什么？

```
cin.clear(0);
cout<<"状态字："<<cin.rdstate()<<endl;
cin.ignore(20,'\n');
```

空读字符，消除上面输入的未被提取的字符和回车

南大学信息科学与工程学院
王文博2026

4. 输入输出成员函数

```
char carr1[20];
cout<<"enter a sentence:"<<endl;
cin.getline(carr1,10,'/');    //指定/为终止字符
cout<<"读入字符串长度为： "<<cin.gcount()<<endl;
cout<<"读入字符串为： "<<carr1<<endl;

if(cin.rdstate()) cin.clear(0);
cin.get(ch);
cout<<"继续读入字符为： "<<ch<<endl;
cout<<"下一个字符是： "<<(char)cin.peek()<<endl; cin.get(ch);
cout<<"继续读入字符： "<<ch<<endl;
cin.putback('0');
cin.getline(carr1,10);
cout<<"读入字符串为： "<<carr1<<endl;
 return 0;
}
```

利用peek函数查看下一个字符，但并不读取

提取当前字符

将字符0插入当前指针所指处

指定换行符为终止字符，读取剩余字符串

4. 输入输出成员函数



问题：

以“时分秒”的格式输入时间，例如输入“12:05:26”提取时、分、秒对应的字符串，如何操作？

5. 重载插入和提取运算符

问题：

用<<和>>可以对基本数据类型进行插入和提取，但是回顾前几次作业里，复数不能直接输出需要一个函数输出实部和虚部两个数据成员，线性表里数据元素是自定义类student、mystring的对象时，打印表元素时，我们重新增加了一个print函数，调用了show函数。

解决方法： C++允许用户重载插入和提取运算符，以方便用户利用输入/输出流来输入输出自定义的数类型，实现对象的对象的输入/输出。

东南大学计算机科学与工程学院
王文博2026

5. 重载插入和提取运算符

例. 通过函数来实现复数相加。

```
#include <iostream>
using namespace std;
class Complex
{
public:
    Complex( ){real=0;imag=0;}
    Complex(double r,double i)
      {  real=r;imag=i;  }
    Complex complex_add(Complex &c2);
    void display( );
 private:
    double real;   //实部
    double imag;  //虚部
};
```

5. 重载插入和提取运算符

```
Complex Complex::complex_add(Complex &c2)
{
    Complex c;
    c.real=real+c2.real;
    c.imag=imag+c2.imag;
    return c;
}
void Complex::display( )
{
   cout<<"("<<real<<","<<imag<<"i)"<<endl;
}
```


东南东南信息技术学程20程学院
王文博2026

5. 重载插入和提取运算符

```
int main( )
{
    Complex c1(3,4),c2(5,-10),c3;
    c3=c1.complex_add(c2);
    cout<<"c1=";
    c1.display( );
    cout<<"c2=";
    c2.display( );
    cout<<"c1+c2=";
    c3.display( );
    return 0;
}
```

调用方式不直观，程序繁琐。

王文博2026

5. 重载插入和提取运算符

对“<<”和“>>”重载的函数形式如下：

```
friend istream & operator >> (istream &, 自定义类 &);
friend ostream & operator << (ostream &, 自定义类 &);
```

友元函数　返回值类型　函数名　左操作数　右操作数

- 重载“>>”的函数的第一个参数和函数的类型都必须是istream&类型。 cin>>a; ⟺ operator>>(cin, a)
- 重载“<<”的函数的第一个参数和函数的类型都必须是ostream&类型。 cout<<a; ⟺ operator<<(cout, a)
- >>和<<重载函数的第二个参数是要进行输出操作的类。
- 一般将重载“>>”和“<<”的函数作为友元或普通函数。

东南大学信息科学与工程学院
王文博2026

5. 重载插入和提取运算符

例：用重载的“>>”和“<<” 输入输出复数

```
#include <iostream>
using namespace std;
class Complex {
  public:
    Complex( ){   real=0;   imag=0;  }
    Complex(double r,double i)
    {  real=r;   imag=i;  }
    Complex operator + (Complex &c2);
    friend ostream& operator << (ostream&, const Complex&);
    friend istream& operator >>  (istream&, Complex&);
private:
   double real;
   double imag;
};
```


东南东南信息技术学程20程学院
王文博2026

5. 重载插入和提取运算符

```
Complex Complex::operator + (const Complex &c2)
{return Complex(real+c2.real,imag+c2.imag); }

ostream& operator << (ostream& output, const  Complex& c)
{
    output<<“(“<<c.real<<“+”<<c.imag<<“i)” <<endl;
    return output;
}

istream& operator >> (istream& input, Complex& c)
{
     cout<<“input real and imaginary  :”;
     input>>c.real>>c.imag;
     return input;
}
```


东南东南信息技术学院 王文博2026

5. 重载插入和提取运算符

```
int main( )
{
        Complex c1,c2;
        cin>>c1>>c2;
        cout<<“c1=“<<c1<<endl;
        cout<<“c2=“<<c2<<endl;
        return 0;
}
```


东南东南信息科学程序设计
王文博2026

5. 重载插入和提取运算符

例：用重载的“<<”输出自定义字符串

```
class mystring
{
        char str[20];
        int maxsize;
        int last;
    public:
        mystring();
        mystring(char *s);
        ~mystring(){}
        friend ostream & operator<<(ostream & ,const mystring &);
        bool operator<(mystring &);
        mystring & operator=(char * ms);
};
ostream & operator<<(ostream & s,const mystring & str)
{
        return s<<cstr.str<<'\t';
}
```


东南东南信息信息科学程序设计学院
王文博2026

5. 重载插入和提取运算符

例：用重载的“>>”和“<<” 输入输出三维点坐标

```
class ThreeDPoint
{
        int x,y,z;
public :
   ThreeDPoint(int a,int b, int c){x=a;y=b;z=c;}
    friend istream &operator>> (istream&input, ThreeDPoint & p);
   friend ostream &operator<<(ostream&output, ThreeDPoint &p);
};
```


东南东南信息科学程序设计
王文博2026

5. 重载插入和提取运算符

```
ostream &operator<< (ostream&output, ThreeDPoint &p)
{
        output<<p.x<<","<<p.y<<","<<p.z<<endl;
        return output;
}
istream &operator>> (istream & input, ThreeDPoint &p)
{
        cout<<“输入x,y,z数值，请以空格间隔:";
         input>>p.x>>p.y>>p.z;
        return input;
}
```

```
int main()
{
        ThreeDPoint P1(10,20,30);
        cout<<P1;
        cin>> P1;
        cout<< P1;
        return 0;
}
```

6. 文件的输入/输出

- 文件的概念
- 文件的打开与关闭
- 对ASCII文件的操作
- 对二进制文件的操作

东南大学信息科学与工程学院
王文博2026

6.1 文件的概念

根据文件中数据的组织形式:

- **ASCII文件(文本文件)**

  由字符序列组成,以ASCII字符形式输入和输出，在文本文件中存取的最小信息单位为字符。

- **二进制文件**

  数据是内存的映像，以字节为单位输入和输出的，在输入和输出时不进行数据格式的转换。

东南大学信息科学与工程学院
王文博2026

6.1 文件的概念

信息在内存中的存放形式:

- **字符数据：ASCII代码**

  存储为ASCII码文件与二进制文件数据一致

- **数值数据：二进制存放**

  两种文件的存储数据形式不一致，字节数不同，内容不同。

字符5：ASCII表示为 00110101

数值 6：二进制表示为 00000110

short类型占2个字节，　　int类型占4个字节

东南大学信息科学与工程学院
王文博2026

6.1 文件的概念

信息在内存中的存放形式:

数据：12345

文本文件中

◆ 当做字符串“12345”，一串ASCII码

➢ ASCII字符串	‘1’	‘2’	‘3’	‘4’	‘5’
➢ 文本流（十六进制形式）	31	32	33	34	35
➢ 文本流（二进制形式）	00110001	00110010	00110101	00110110	00110101

东南大学信息科学与工程学院
王文博2026

6.1 文件的概念

信息在内存中的存放形式:

数据：12345

二进制文件中

◆ 12345若以int型存储，占4个字节

➢ 内存中int数据的形式

00000000	00000000	00110000	00111001

➢ 二进制流中的形式

00000000	00000000	00110000	00111001

东南大学信息科学与工程学院
王文博2026

6.2 文件打开与关闭

1.打开磁盘文件:

(1) 为文件流对象和指定的磁盘文件建立关联。

在程序内定义一个文件类的对象，由该对象与文件发生联系，程序内所有的与文件的操作都是对该对象的操作。

```
fstream    iofile;
ifstream   infile;
ofstream   outfile;
```

对象可联系输入输出文件

对象只能联系输入文件

对象只能联系输出文件

东南大学信息科学与工程学院
王文博2026

6.2 文件打开与关闭

(2) 调用成员函数open打开文件，并可指定文件的工作方式： 输入、输出、ASCII或二进制文件等。

调用成员函数open的一般形式为：

文件流对象 . open(磁盘文件名，输入输出方式);

```
infile.open ("f1.dat", ios::in);
outfile.open("f1.dat", ios::out);
iofile.open ("f1.dat", ios::in|ios::out);
iofile.open ("f1.dat", ios::in);
iofile.open ("f1.dat", ios::out);
```

```
infile.open("f1.dat");
默认打开方式输入
```

```
outfile.open("f1.dat");
默认打开方式输出
```

```
iofile.open("f1.dat");
默认打开方式可输入输出
```

磁盘文件名可以包括路径，如：`"c:\\new\\f1.dat"`**，缺省路径，默认为当前目录下的文件。目录需要已存在！**

东南大学信息科学与工程学院
王文博2026

6.2 文件打开与关闭

三个文件流类都重载了一个带默认参数的构造函数，功能与open一样，所以也可以在定义文件流对象时指定参数,调用文件流类的构造函数来实现打开文件的功能：

文件流类 对象名(磁盘文件名，输入输出方式);

如：

```
ofstream outfile ("f1.dat", ios::out);
or  ofstream outfile ("f1.dat");
    fstream iofile("f1.dat", ios::in|ios::out) ;
or  fstream iofile("f1.dat) ;
```


东南东南信息科学程序设计
王文博2026

6.2 文件打开与关闭

ios类中关于文件打开方式定义的枚举成员说明:

```
enum _Openmode
{// constants for file opening options
 _Openmask = 0xff};
static const _Openmode in = (_Openmode)0x01;
static const _Openmode out = (_Openmode)0x02;
static const _Openmode ate = (_Openmode)0x04;
static const _Openmode app = (_Openmode)0x08;
static const _Openmode trunc = (_Openmode)0x10;
static const _Openmode _Nocreate = (_Openmode)_IOS_Nocreate;
static const _Openmode _Noreplace = (_Openmode)_IOS_Noreplace;
static const _Openmode binary = (_Openmode)_IOSbinary;
```

打开文件用于输入操作（从文件读取）

打开文件用于输出操作（写入文件）

打开文件，文件指针在文件尾，但文件指针可移动，即新数据可写到任何位置

打开文件用于输出，原文件内容保留，新数据接在尾部

打开文件，并清空它

不创建新文件

不替换方式

以二进制方式打开文件

东南大学信息科学与工程学院
王文博2026

6.2 文件打开与关闭

- 可以用“位或”运算符“|”对输入输出方式进行组合。

ios::in //以读方式打开一个文本文件， 若文件不存在，打开失败

ios::out //以写方式打开一个文本文件，若文件不存在，创建一个
若文件存在，清除原内容

ios::in | ios::out //以读写方式打开一个文本文件， 若文件不存在，打开失败

ios::in | ios::out | ios::binary //以读写方式打开一个二进制文件

ios::out | ios::app //以追加写方式打开一个文本文件

ios::out | ios::app | ios::binary //以追加写方式打开一个二进制文件

ios::out | ios::Nocreate //以写方式打开一个文本文件，若文件存在打开成功，若文件不存在，打开不成功，不创建新文件

6.2 文件打开与关闭

- 打开输入文件时，文件必须存在!!!!

- 打开文件应判断是否成功，若打开操作失败，open函数返回0 (假)值,若采用构造函数的方式打开文件，则流对象的值为0。

```
infile.open("f1.dat",ios::in);
if (!infile)  {
 cout<<"open error";
  exit(1);
}
```

- 每一个打开的文件都有一个文件指针, 起始位置由输入输出方式决定。

不特别指明时，一般打开文件时，指针在文件首部，进行提取和插入操作时，指针自动向后移动。

东南东南信息技术学院 王文博2026

2. 关闭磁盘文件:

关闭文件用成员函数close。

如:　outfile. close( );

解除该磁盘文件与文件流的关联，不能再通过文件流对该文件进行输入或输出。

注意关闭文件并没有取消文件流对象，该文件流对象又可与其他磁盘文件建立联系。

文件流对象在程序结束时，或它的生命期结束时，由析构函数撤消 。

东南东南信息科学程2024学院
王文博2026

6.2 文件打开与关闭

文件打开关闭过程小结：

※ 声明文件流对象；

※ 用文件流对象打开一个文件(可创建): 将文件流类与某磁盘文件联系起来；

※ 使用文件流类提供的成员函数:将数据写入到文件中或从文件中读取数据；

※ 关闭已打开的文件:将该磁盘文件与文件流类已建立的关系脱离。



东南大学信息科学与工程学院
王文博2026